

I. PILE D'APPELS – ARBRE DES APPELS

Exo 1. Exécuter à la main les fonctions des exemples de 1 à 4 du fichier [exemple_pile_des_appels.py](#)

Indiquer pour chaque exemple les résultats attendus dans la variable `le_resultat`.

Vérifier ces résultats en lançant Python sur machine.

Exo 2. En s'aidant de l'application en ligne <http://vps339729.ovh.net/#/>, permettant la visualisation de *la pile d'appels*, on examinera pour chacun des exemples 1 à 3 figurant dans le fichier [exemple_pile_des_appels.py](#) les appels successifs et les états successifs de la pile à chaque appel et retour de fonctions.

Remplir la fiche suivante de l'arbre des appels [SY Arbres et Piles des Appels.docx](#)

L'ensemble des informations sauvegardées (valeurs des arguments, valeur des variables locales, adresse de retour) dans la pile à l'appel d'une fonction constitue le contexte d'exécution de la fonction.

Exo 3. De manière analogue à ce qui a été fait à l'exemple #3, dessiner l'arbre des appels et donner les états successifs de la pile à chaque appel pour les exemples #4 et #5.

II. RECURSIVITE

Exo 4. De manière analogue, traiter les exemples #1 et #2 du fichier [exemple_pile_des_appels_recurifs.py](#)



La récursivité est un principe de pensée exigeant et parfois vertigineux, ...

1. LA RECURSIVITE EXISTE VRAIMENT

a) Dans les arts :

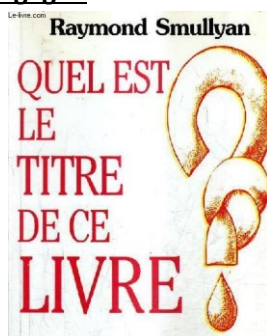


Artiste néerlandais **MAURITS CORNELIS ESCHER (1898 – 1972)**



PINK FLOYD - « UMMAGUMMA » (1969)

b) Dans le langage :



c) Dans la pub



2. FONCTIONS RECURSIVES

Une fonction est qualifiée de **récursive** si elle s'appelle elle-même.

Et pour laisser une chance au processus d'appels successifs de s'arrêter, cet appel à soi-même doit être conditionné : on stoppe l'appel à soi-même lorsque certaines conditions sont réunies.



Une fonction récursive sans condition d'arrêt ne s'arrête donc jamais. En pratique Python prévoit une profondeur de récursion maximum (par défaut 1000, mais modifiable), l'atteindre provoque une erreur, et surtout témoigne probablement d'une condition d'arrêt défectueuse...



Écrire une fonction sous forme récursive est souvent plus naturel. Cependant la récursivité est un principe de pensée exigeant et parfois vertigineux...

Rq 1. Tout algorithme itératif possède une version récursive, et réciproquement

Exo 5. Exemple du calcul de la factorielle de n défini par $n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$, et le cas particulier: $0! = 1$ (par convention)

En utilisant le fichier `factorielle_tbc.py` à compléter :

- Écrire une version itérative de la fonction factorielle dont on a rencontré plus haut une version qui renvoie la factorielle d'un entier n donné.
- Écrire une fonction `genere_tab(N, max)` qui retourne une liste de N entiers aléatoirement choisis entre 0 et max.
- Calculer les factorielles des éléments du tableau par chacune des deux méthodes, stocker les résultats des calculs, mesurer les deux temps d'exécution. Proposer une explication à cette contre-performance.



[factorielle.py](#)

3. FORMULATION RECURSIVE



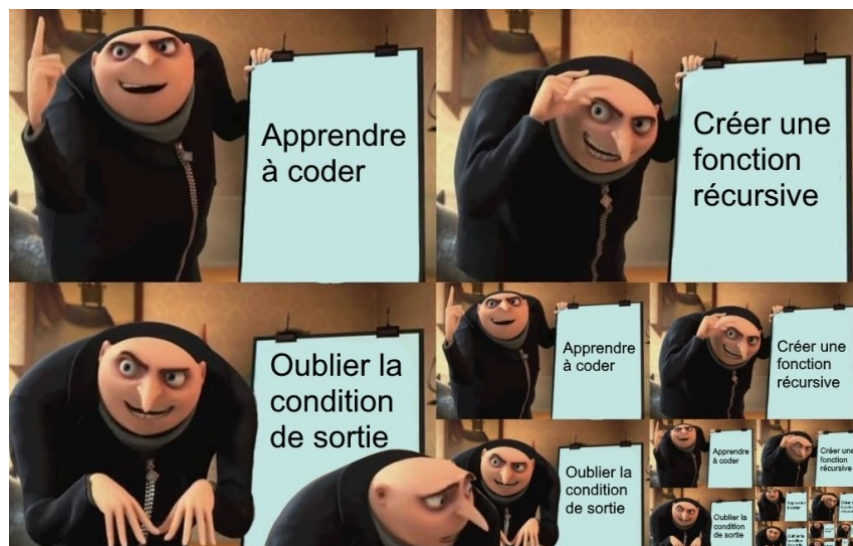
De la même manière que l'écriture $n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$, ou la définition précisant en langage naturel que $n!$ est le produit des n premiers entiers strictement positifs orienté vers une version itérative de la fonction factorielle, la version récursive sera associée à la formulation suivante :

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \times \dots & \text{si } n \neq 0 \end{cases} \quad \text{car} \quad n! = n \times \underbrace{(n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1}_{(n-1)!}$$

Cette formulation fait apparaître la condition d'arrêt et l'appel récursif de **factorielle(n)** à **factorielle(n-1)**

Rq 2. Un appel de fonction est une opération plus coûteuse en soi que, par exemple, une opération arithmétique ou un test. C'est pourquoi on préfère souvent la fonction itérative à la version récursive. Dans le cas de la factorielle, on prendra plutôt la version itérative, mais il y a des cas où la fonction récursive est clairement préférable, par exemple pour parcourir des arbres.

Rq 3. En revanche, les grandes profondeurs de récursion peuvent solliciter fortement la mémoire et rendre pour le moins son exécution plus lente (plus radical, le débordement de pile (*stack overflow*) est fatal au programme)



Exo 6. Donner une formulation récursive et implémenter une version récursive des fonctions suivantes :

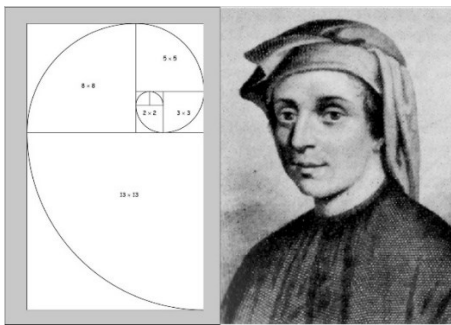
- puissance(x,n)** qui renvoie x^n , où x est un flottant et n un entier positif.
- somme(n)** qui renvoie la somme des n premiers entiers.
- sommeGeo(n,u0,q)** qui renvoie la somme des n premiers termes d'une suite géométrique de raison q et de premier terme u_0 .

Pour les tests on pourra insérer l'assert suivant à compléter en utilisant la formule de la somme des termes d'une suite géométrique :

```
nb_termes=5;u0=1;q=-7;
assert sommeGeo(nb_termes, u0 , q)==
```

4. RECURSIVITE DOUBLE

Exo 7. La suite de **Leonardo Fibonacci** est définie pour tout $n \in \mathbb{N}$ par

$$\begin{cases} u_0 = 1; \\ u_1 = 1 \\ u_{n+2} = u_{n+1} + u_n \end{cases}$$


On connaît les deux premiers termes u_0 et u_1 , et chaque nouveau terme est la somme des deux précédents

- La définition par récurrence donnée ci-dessus est proche d'une formulation récursive de la fonction **fibonacci(n)** qui renvoie le terme de rang n :

$$fibonacci(n) = \begin{cases} & \text{si } n = 0 \\ & \text{si } n = 1 \\ & \text{si } n > 1 \end{cases}$$

- Implémenter une version récursive de la fonction **fibonacci(n)**
- Tracer l'arbre des appels de l'exécution de **fibonacci(5)**.
- Que peut-on penser de cette implémentation en termes de performance ?
- Le code de la fonction **leonardo(n)** ci-dessous, supposé renvoyer le terme de rang n de la suite de Fibonacci, améliore-t-il ou dégrade-t-il la version précédente ?

```
def leonardo_f(n):
    def fib_rec(a, b, n):
        if n == 0:
            return a
        else:
            return fib_rec(b, a+b, n-1)
    return fib_rec(0, 1, n)
```



Confirmer sur machine les performances attendues de chaque version en utilisant la fonction **timeit.timeit()** (import nécessaire de la librairie **timeit**), qui mesure le temps de calcul CPU pour l'exécution passée en paramètre :

```
import timeit
print("bench imperative :",
      timeit.timeit(lambda: fibo(35), number=1))
print("bench recursive double :",
      timeit.timeit(lambda: fibo_rec_db(35), number=1))
print("bench recursive simple :",
      timeit.timeit(lambda: fibo_rec_simple(35), number=1))
```

 fibonacci.py

III. Exos

Exo 8. Écrire une fonction récursive **nombre_de_chiffres (n)** qui prend un entier positif ou nul **n** en argument et renvoie son nombre de chiffres.
Par exemple **nombre_de_chiffres (34126)** renvoie 5

 [arithmetique.py](#)

Exo 9. Écrire une fonction récursive **pgcd (a,b)** calculant le PGCD de deux entiers positifs a et b (non tous deux nuls) par l'algorithme d'Euclide.

 On rappelle l'algorithme d'Euclide (cf Maths de 3^{ème}) dans sa version itérative :

```
PGCD(a,b) :
    tant que b > 0, faire
        r ← a%b
        a ← b
        b ← r
    Fin tant que
    Retourner a
```

 Cet algorithme repose sur le lemme d'Euclide suivant :

Lemme 1. *a* et *b* sont des naturels non nuls.
si les entiers *q* et *r* sont tels que $a = bq + r$ avec $0 \leq r < b$,
Alors $\text{PGCD}(a;b) = \text{PGCD}(b;r)$

 [arithmetique.py](#)

Exo 10. On donne le code suivant :

```
def puissance_logn(x,n):
    if n==0 :
        return 1
    elif n==1 :
        return x
    else:
        r=puissance_logn(x,n//2)
        if n%2 ==0:
            return r*r
        else:
            return x*r*r
```

- Quelle est la valeur retournée par **puissance_logn(3,4)** ?
- Dresser l'arbre des appels effectués lors de l'exécution de **puissance_logn(3,9)**
- Comparer les performances respectives de la fonction **puissance(x,n)** vue plus haut et de la fonction **puissance_logn(x,n)** lorsque n est grand.

- d. A combien d'appels récursifs successifs de la fonction **puissance_logn()** va donner lieu l'exécution de **puissance_logn(3,1024)** ? **puissance_logn(3,2**25)** ? **puissance_logn(3, 2**p)** ?



Si $n = 2^p$, alors $p = \log_2(n)$

L'exécution de **puissance_logn(3, n)** donne donc lieu à $p = \log_2(n)$ appels successifs de la fonction **puissance_logn(3, n)**. On parle alors de complexité en $O(\log(n))$.

Rq 4. En comparaison la complexité de la fonction **puissance(x,n)** vue plus haut est en $O(n)$.

Exo 11.

- Ecrire une fonction **carre_diviseur(A)** qui retourne le plus petit entier $c > 1$ dont le carré divise **A**, nombre entier carré parfait.
- Écrire une fonction récursive **racine_carre_parfait (A)** qui renvoie la racine carrée d'un entier **A** carré parfait.
Par exemple **racine_carre_parfait (7056)** renvoie **84**

On pourra utiliser l'égalité à compléter suivante : $\sqrt{a} = c \times \sqrt{\frac{\dots}{\dots}}$



[racine_carre_parfait_recursive.py](#)

Exo 12. Le triangle de Pascal

- Coefficients $\binom{n}{p}$

Le triangle de Pascal affiche les coefficients binomiaux $\binom{n}{p}$ à la $n^{\text{ème}}$ ligne et à la $p^{\text{ème}}$ colonne.

(la numérotation de n et p commence à 0)

	p	0	1	2	3	4	5	6	7	8	9	10
n												
0 :		1										
1 :		1	1									
2 :		1	2	1								
3 :		1	3	3	1							
4 :		1	4	6	4	1						
5 :		1	5	10	10	5	1					
6 :		1	6	15	20	15	6	1				
7 :		1	7	21	35	35	21	7	1			
8 :		1	8	28	56	70	56	28	8	1		
9 :		1	9	36	84	126	126	84	36	9	1	
10 :		1	10	45	120	210	252	210	120	45	10	1

- Donner une formulation récursive de la fonction **C(n,p)** qui renvoie le coefficient $\binom{n}{p}$:

$$C(n, p) = \begin{cases} 1 & \text{si} \\ \text{sinon} & \end{cases}$$

- Implémenter une version récursive de la fonction **C(n,p)**
- Dessiner le triangle de Pascal avec deux boucles **for** en faisant varier n entre 0 et 10



[triangle_de_pascal.py](#)

Exo 13. Flocon de VonKoch[TD Récursivité VonKoch.docx](#)**Exo 14.** Triangle de Sierpinski[TD Récursivité Sierpinski.docx](#)**Exo 15.** Robot

L'Agence Spatiale a perdu le contact avec son robot d'exploration sur la planète Mars. En l'absence de directive, ce dernier continue son exploration et se déplace selon le protocole suivant. Chaque jour, il :

- choisit une direction au hasard entre quatre : Nord, Sud, Est, Ouest ;
- avance d'une unité de distance dans cette direction.

La direction choisie un jour ne dépend pas des directions choisies les jours précédents.

1. Écrire une fonction récursive **déplacements(n)** qui simule les déplacements du robot et donne les coordonnées du site atteint au bout de n jours.
2. Écrire une fonction récursive **nombre_de_chemins(n,x,y)** qui calcule le nombre de chemins possibles pour atteindre le site de coordonnées $e(x, y)$ ou x et z sont des entiers, en n jours.

Exo 16.

Pour convertir un nombre exprimé en base 10 en binaire, on procède par divisions par deux successives jusqu'à obtenir un nombre inférieur à 2. L'écriture binaire est alors les restes obtenus écrits du **dernier** au **premier**.

Ecrire en python, une fonction binaire récursive, de paramètre un entier n positif, qui renvoie, sous forme de chaîne, l'écriture de n en binaire.

Ainsi : **binaire(13)** renvoie "1101".

[binaire.py](#)